

REPORTE DE PRÁCTICA: ARREGLOS UNIDIMENSIONALES Y MEMORIA CONTIGUA

Asignatura: Programación Estructurada
Estudiante: Aldo Ezequiel Rodriguez Godoy
Código: 224097617
Sección: D04
Profesor: JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha: 6 de marzo del 2026

INTRODUCCIÓN

En el estudio de las estructuras de datos dentro del lenguaje C, un arreglo unidimensional (también conocido como vector o matriz unidimensional) representa una colección finita y homogénea de elementos ordenados. Decimos que es homogénea porque todos los datos almacenados en su interior comparten estrictamente el mismo tipo de dato, y es ordenada porque cada elemento individual posee un índice numérico único que define su posición, comenzando siempre desde la posición cero.

A nivel físico y de hardware, el concepto más crucial de un arreglo es que las variables se localizan en la memoria RAM de manera contigua. Esto significa que el compilador reserva un bloque de celdas adyacentes, una inmediatamente detrás de la otra. La cantidad de bytes que separa a un elemento del siguiente equivale exactamente al tamaño físico del tipo de dato declarado (por ejemplo, 4 bytes para enteros e identificadores float estándar).

Un aspecto teóricamente relevante del lenguaje C es el comportamiento del identificador del arreglo. El nombre del vector funciona internamente como un puntero constante apuntando hacia la dirección física del primer elemento (índice 0). En esta actividad exploramos empíricamente este comportamiento imprimiendo los mapas de direccionamiento de los vectores, comprobando su secuencialidad y, posteriormente, aplicando este conocimiento para construir un capturador y lector iterativo de arreglos dinámicos mediante bucles for.

OBJETIVO

Objetivo General

Construir, analizar e implementar código en lenguaje C enfocado en arreglos unidimensionales para comprender de forma empírica la distribución física contigua que sufren los vectores en la memoria RAM y dominar la lectura y escritura iterativa de datos flotantes.

Objetivos Particulares

- Analizar y comparar las direcciones de memoria obtenidas mediante el identificador directo del arreglo y la extracción explícita del índice cero para corroborar su equivalencia sintáctica.
- Diseñar e implementar estructuras de repetición for parametrizadas para realizar la captura interactiva y almacenamiento de datos de temperatura en un vector.

- Recorrer de manera secuencial los índices de un arreglo flotante para recuperar y mostrar en la consola de salida los valores previamente resguardados.

DESARROLLO

Para la realización de esta práctica, el trabajo de programación de laboratorio se dividió claramente en dos fases lógicas separadas, cada una orientada a estudiar una naturaleza distinta de los arreglos:

Fase 1: Análisis de Direccionamiento de Hardware

En la primera etapa de la actividad, se compiló un bloque de código diseñado para imprimir las ubicaciones físicas de diferentes variables en la pila. Declaré un arreglo contiguo de 24 elementos flotantes llamado `temp`, dos variables flotantes independientes (`temp1` y `temp2`), y una variable entera de tipo `dummy`. Mediante el uso de `printf`, extraje sus celdas de asignación.

Lo que se buscaba demostrar con esto es el comportamiento sintáctico de los arreglos. Al imprimir la dirección de la etiqueta del arreglo (`&temp`), el valor hexadecimal arrojado es exactamente igual a la celda que obtenemos al extraer explícitamente el primer elemento (`&temp[0]`). Esto comprueba que el nombre del arreglo apunta directamente al inicio del bloque asignado.

Fase 2: Manipulación Iterativa de Datos

En la segunda fase del ejercicio, pasamos a la parte funcional de la manipulación de vectores. Definí un límite de control entero `N` con valor de 4, y declaré un arreglo de temperaturas flotantes de tamaño `N`. Posteriormente, programé dos ciclos secuenciales `for`:

- **Ciclo de Escritura (Captura):** El primer bucle corre desde el índice `i = 0` hasta `i < N`. En cada iteración, el programa solicita al usuario una temperatura por consola y la guarda en la celda de memoria contigua mediante la instrucción `scanf("%f", &temp[i])`.
- **Ciclo de Lectura (Impresión):** El segundo bucle repite el recorrido secuencial, pero esta vez lee el contenido de cada índice y lo muestra impreso de forma ordenada en la terminal.

Los códigos desarrollados y analizados durante la práctica se detallan a continuación:

```
// --- CODIGO 1: ANALISIS DE MAPAS DE DIRECCIONAMIENTO ---
#include <stdio.h>
#include <stdlib.h>

int main(){
    float temp[24];
    float temp1, temp2;
    int dummy;

    printf("direccion de mem de dos flotantes: &temp1: %i, \n&temp2 %i!\n", &temp1, &temp2);
    printf("\ndireccion de temp[24]: %i,\n", &temp);
    printf("\ndireccion de &temp[24]: %i,\n", &temp[0]);
    printf("\ndireccion de &dummy:%i\n", &dummy);

    return 0;
}

// --- CODIGO 2: CAPTURA Y RECORRIDO ITERATIVO ---
#include <stdio.h>
#include <stdlib.h>

int main(){
    int N=4;
    float temp [N];
    int i;

    for( i=0 ; i<N; i++){
        printf("ingresa la temperatura de %d:\n", i);
        scanf("%f", &temp[i]);
    }

    for( i=0 ; i<N; i++){
        printf("las tempreraturas ingresadas fueron:%f\n", temp[i]);
    }

    return 0;
}
```

CONCLUSIÓN

El desarrollo de esta actividad de laboratorio fue de gran ayuda para terminar de entender cómo se comportan los arreglos en el bajo nivel. Muchas veces usamos los arreglos de manera automática en otros lenguajes pensando que son listas mágicas, pero al trabajar en C y poder imprimir sus direcciones de memoria físicas, se vuelve evidente que un arreglo no es más que un bloque continuo de bytes reservado en la RAM.

Lo que más me llamó la atención fue confirmar en la primera fase que imprimir el nombre del arreglo da exactamente el mismo resultado numérico que imprimir la dirección de su primer elemento. Esto me aclaró la teoría vista en clase de que el identificador funciona como un apuntador constante al inicio de la estructura. Además, al realizar la segunda fase con temperaturas flotantes, me di cuenta de lo cómodo que es procesar información repetitiva usando ciclos for parametrizados, ya que el índice del contador del ciclo coincide perfectamente con la posición de la celda de datos que queremos modificar. Este conocimiento de cómo se estructuran las direcciones de los arreglos me da bases sólidas para entender el uso de arreglos multidimensionales (matrices) y cadenas de caracteres en las próximas prácticas de la materia.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Gottfried, B. S. (2005). *Programación en C* (2da ed.). McGraw-Hill.